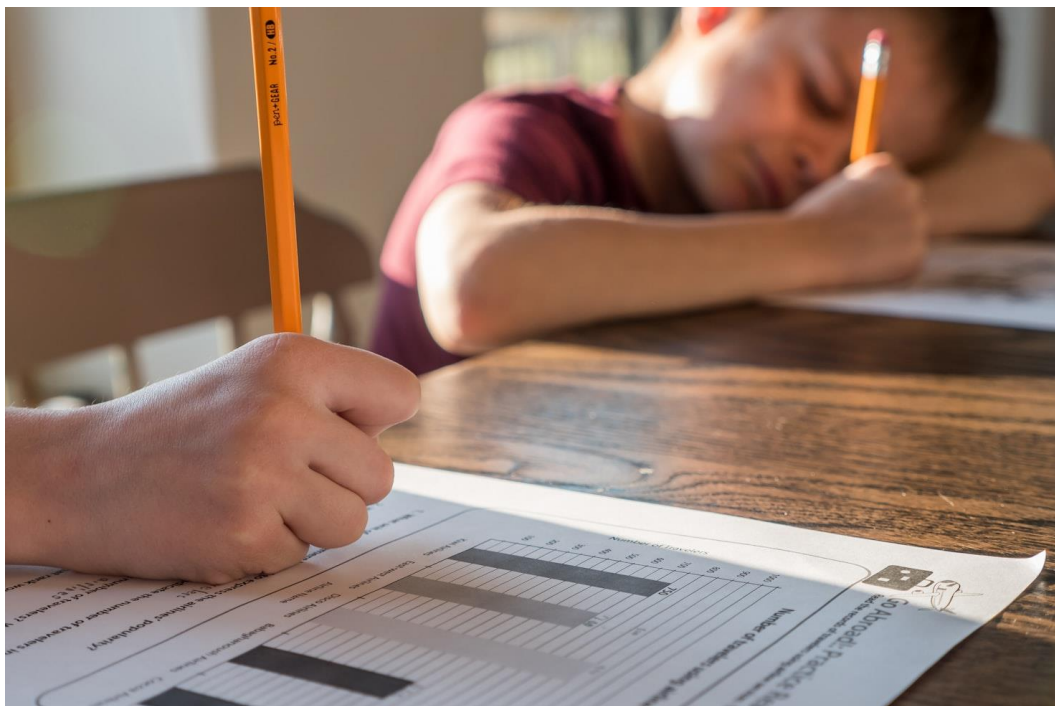


CFC Informaticien

Travail Pratique Individuel

Kiddo

Suivi des devoirs des enfants



Candidat	<i>Almeida Nelson</i>
École professionnelle / Entreprise formatrice	École technique des métiers de Lausanne
Chef de projet	<i>Schaffter Cédric</i>
Expert 1	<i>Oberson Bernard</i>
Expert 2	<i>Maret Gabriel</i>
Date de début	07.05.2026
Date de fin	01.06.2026

Table des matières

1 Résumé	4
2 Glossaire.....	4
3. Rapport de Projet.....	4
3.1 Contexte et mandat	4
3.2 Méthode et organisation	4
3.3 Analyse des exigences.....	5
3.3.1 Exigences fonctionnelles	6
3.3.2 Exigences non fonctionnelles.....	7
3.4 Gestion des risques et mesures.....	8
3.4.1 Contexte de sécurité analysé.....	8
3.4.2 Tableau des risques et mesures	8
3.4.3 Intégration des mesures dans les exigences	8
3.5 Conception.....	9
3.5.1 Maquettes basse fidélité	9
3.5.2 Maquettes haute fidélité et identité visuelle	9
3.5.3 Parcours utilisateur parent	10
3.5.4 Gestion de la famille et des comptes enfants.....	11
3.5.5 Parcours utilisateur enfant	12
3.5.6 Architecture générale	15
3.5.7 Choix de conception du modèle de données	15
3.5.8 Modèle relationnel retenu.....	16
3.5.9 Rôles, droits d'accès et cohérence métier.....	18
3.5.10 Contraintes SQL et valeurs contrôlées.....	19
3.6 Planification et suivi	20
3.7 Réalisation	20
3.7.1 Mise en place de l'environnement backend.....	20
3.7.2 Implémentation de la base de données et des migrations	20
3.7.3 Authentification et sécurité backend	20
3.7.4 Implémentation des fonctionnalités métier	21
3.7.5 Corrections et ajustements après tests Bruno	21
3.8 Tests et validation	24
3.8.1 Stratégie et environnement de test.....	24
3.8.2 Tests prévus	26
3.8.3 Exécution automatique dans le CI/CD	26

3.8.4 Exécution et résultats des tests.....	26
3.9 Livraison	27
3.10 Conclusion et bilan personnel.....	27
4 Références.....	27
5 Annexes	27
5.1 Journal de travail.....	27
5.2 Liste des livrables remis	27
5.3 Repos GitHub	27
5.4 Table des illustrations	27

1 Résumé

2 Glossaire

3. Rapport de Projet

3.1 Contexte et mandat

L'idée derrière Kiddo est assez simple au premier abord : offrir aux parents un outil pour suivre les devoirs et les tâches de leurs enfants, directement depuis une application web. Mais en y regardant de plus près, le projet s'avère nettement plus riche que ça.

L'application doit s'adapter à des configurations familiales variées : une famille peut avoir plusieurs parents, plusieurs enfants, ou les deux. Et tous ces utilisateurs n'ont pas les mêmes droits. Un parent peut créer des tâches, les modifier et valider le travail réalisé. Un enfant, de son côté, voit uniquement ce qui lui est attribué, peut signaler qu'une tâche est terminée et joindre une preuve à l'appui.

Ce qui rend le mandat particulièrement intéressant, c'est qu'il ne s'arrête pas à la simple gestion de listes. Il faut réfléchir à des situations potentiellement conflictuelles : que se passe-t-il, par exemple, si deux parents interviennent en même temps sur la même tâche ? L'un la modifie pendant que l'autre tente de valider une preuve. Ce genre de cas limite doit être anticipé dès la conception et au moins un scénario concret et reproductible doit en faire la démonstration. Sur le plan technique, les attentes sont claires : une base de données SQL, une interface responsive, une attention particulière aux failles de sécurité référencées dans l'OWASP Top 10, une couverture de tests automatisés, et un déploiement en production sur Azure piloté par un workflow GitHub Actions.

3.2 Méthode et organisation

Pour ce TPI, j'ai choisi une méthode de projet séquentielle structurée, adaptée à un travail individuel limité à 90 heures. Le cahier des charges étant défini dès le départ, il est nécessaire d'organiser le travail de manière claire afin de garder une vue d'ensemble sur les exigences, les livrables, les tests, la documentation et le déploiement.

Cette méthode repose sur une planification initiale, un suivi régulier de l'avancement et une documentation progressive des décisions prises. Les différentes activités du projet ne sont pas traitées comme des blocs totalement séparés : certaines tâches, comme les tests ou la documentation, accompagnent aussi la conception et l'implémentation afin d'éviter de tout repousser à la fin.

La méthode n'est donc pas appliquée de manière rigide. Des ajustements restent possibles grâce au journal de travail et au suivi réel du planning. L'objectif est de comparer régulièrement le travail prévu avec le travail effectivement réalisé, d'identifier les écarts et de documenter les décisions prises pendant le projet.

Cette organisation permet de limiter les risques principaux du TPI : commencer l'implémentation sans analyse suffisante, repousser les tests trop tard ou négliger la documentation. Les tâches ont été réparties selon les proportions officielles du mandat : analyse, implémentation, tests et documentation.

3.3 Analyse des exigences

L'analyse du cahier des charges a permis d'identifier les exigences fonctionnelles, les exigences non fonctionnelles, les contraintes techniques et les livrables attendus. Les exigences fonctionnelles décrivent ce que l'application doit permettre aux utilisateurs de faire. Les exigences non fonctionnelles décrivent les qualités attendues de la solution, notamment en matière de sécurité, de cohérence des données, de tests, de déploiement et de documentation.

Une attention particulière a été portée aux rôles parent et enfant, car ils déterminent les droits d'accès aux tâches, aux preuves et aux actions disponibles dans l'application. Les règles de visibilité et de modification sont centrales dans ce projet, car elles influencent directement le modèle de données, les contrôles backend et les cas de test.

3.3.1 Exigences fonctionnelles

ID	Exigence fonctionnelle	Détails couverts	Priorité
EF-01	Gérer les comptes utilisateurs	Création de comptes parent et enfant	Haute
EF-02	Gérer les familles	Ajout d'un parent ou enfant dans une famille via invitation applicative, avec acceptation ou refus par l'utilisateur invité.	Haute
EF-03	Gérer les rôles parent/enfant	Vues et informations différentes selon le rôle	Très haute
EF-04	Permettre au parent de gérer les tâches	Création, consultation, modification et suppression des tâches selon les règles définies	Très haute
EF-05	Permettre au parent de définir les droits d'accès aux tâches	Accès des autres parents : modification, lecture seule ou aucun accès	Très haute
EF-06	Permettre au parent de valider ou refuser une tâche terminée	Validation/refus basé sur une preuve fournie par l'enfant	Très haute
EF-07	Permettre à l'enfant de consulter ses tâches	L'enfant voit uniquement les tâches qui lui sont attribuées et le parent assignant	Très haute
EF-08	Permettre à l'enfant de terminer une tâche	Marquer une tâche comme terminée et fournir une preuve photo ou texte	Haute
EF-09	Permettre à l'enfant de gérer ses propres tâches	Création, modification, suppression et visibilité des tâches créées par l'enfant	Haute
EF-10	Permettre le tri des tâches	Tri par ordre alphabétique ou par date d'échéance	Moyenne
EF-11	Gérer les conflits métier	Validation concurrente, modification après validation, droits incohérents	Très haute

3.3.2 Exigences non fonctionnelles

ID	Exigence non fonctionnelle	Détails couverts	Priorité
ENF-01	Interface responsive et utilisable	L'application doit être utilisable sur PC et mobile, avec une navigation claire et une interface cohérente.	Haute
ENF-02	Sécurité des comptes	Les comptes locaux doivent être protégés, notamment contre les attaques par rainbow table grâce au hachage sécurisé des mots de passe.	Très haute
ENF-03	Contrôle des rôles et autorisations	Les rôles parent/enfant et les droits d'accès aux tâches doivent être vérifiés côté backend.	Très haute
ENF-04	Validation des données	Les entrées utilisateur doivent être validées côté backend afin d'éviter les données invalides ou manipulées.	Très haute
ENF-05	Protection contre l'injection SQL	L'accès aux données doit se faire de manière sécurisée, par exemple avec un ORM ou des requêtes paramétrées.	Très haute
ENF-06	Gestion sécurisée des erreurs	Les messages d'erreur ne doivent pas exposer d'informations techniques sensibles.	Haute
ENF-07	Cohérence des données	Le système doit empêcher les états incohérents, notamment lors des validations, refus, modifications ou conflits entre utilisateurs.	Très haute
ENF-08	Base de données SQL et scripts	Une base SQL doit être utilisée avec setup_database.sql pour le schéma et init_database.sql pour les données de test.	Très haute
ENF-09	Tests automatisés	Le projet doit inclure au moins 10 tests unitaires, 1 test d'intégration, et leur exécution dans le CI/CD.	Très haute
ENF-10	Déploiement automatisé	L'application doit être déployée sur Azure via un workflow GitHub Actions.	Très haute
ENF-11	Gestion des secrets et de la configuration	Les variables sensibles, la configuration CORS et les paramètres d'environnement ne doivent pas être exposés ou ouverts inutilement.	Très haute
ENF-12	Gestion des dépendances	Les dépendances utilisées doivent être limitées aux besoins du projet et vérifiées afin de réduire les risques liés aux bibliothèques vulnérables ou inutiles.	Moyenne

3.4 Gestion des risques et mesures

3.4.1 Contexte de sécurité analysé

L'analyse de sécurité est basée sur l'environnement prévu pour l'application Kiddo. Le système est composé d'une interface web utilisée par les parents et les enfants, d'une API backend responsable des règles métier et des contrôles d'accès, et d'une base de données SQL stockant les comptes utilisateurs, les familles, les tâches, les droits et les preuves.

Les risques ont été identifiés à partir du cahier des charges, des rôles utilisateurs, des accès aux données et des risques courants présentés par l'OWASP Top 10. L'objectif est de prévoir les mesures de sécurité dès la conception, afin que les contrôles soient appliqués dans le code de l'application, principalement côté backend.

Élément	Description	Risque principal
Interface web parent	Dashboard parent, tâches, validations, invitations	Accès à des actions non autorisées
Interface web enfant	Consultation des tâches, preuve, tâches créées par l'enfant	Accès à des tâches d'un autre enfant
API backend	Authentification, rôles, règles métier, validation	Contournement du frontend par requêtes directes
Base SQL	Comptes, familles, tâches, preuves, droits	Fuite ou incohérence des données
GitHub / CI-CD	Code source, workflows, secrets	Exposition de secrets ou mauvaise configuration
Azure	Frontend, backend, base de données en production	Mauvaise configuration d'environnement

3.4.2 Tableau des risques et mesures

Le tableau détaillé des risques et des mesures de sécurité est placé en annexe afin de ne pas surcharger le rapport principal. Cette annexe présente, pour chaque risque identifié, la manipulation ou le problème possible, l'impact, la mesure prévue, l'exigence liée et le moyen de vérification prévu.

Les risques sont regroupés par catégories afin de garder une vue d'ensemble claire. Les risques les plus importants concernent principalement les droits d'accès, la cohérence des données, la sécurité des comptes, la validation des entrées, la gestion des erreurs, la protection des secrets et la configuration de l'environnement de production.

Les mesures prévues seront prises en compte dans la conception du modèle de données, dans les contrôles backend, dans les exigences non fonctionnelles et dans les futurs cas de test.

Le tableau complet est disponible en *Annexe 2 : Tableau détaillé des risques et mesures de sécurité*.

3.4.3 Intégration des mesures dans les exigences

Les mesures de sécurité identifiées sont intégrées aux exigences non fonctionnelles, notamment pour la gestion des rôles côté backend, le contrôle d'accès, la validation des entrées, la protection contre l'injection SQL, le hachage des mots de passe, la

gestion des erreurs et la protection des secrets. Elles seront également reprises dans les cas de test afin de vérifier que les mesures prévues sont effectivement appliquées.

3.5 Conception

La phase de conception a pour objectif de définir la structure générale de l'application avant le développement. Elle permet de clarifier les écrans principaux, la navigation, les rôles parent/enfant et les actions disponibles selon le type d'utilisateur.

3.5.1 Maquettes basse fidélité

Les premières maquettes basse fidélité ont été réalisées afin de définir la structure générale de l'application avant de travailler l'aspect visuel détaillé. Elles ont permis de clarifier les écrans principaux, la navigation, la séparation entre l'espace parent et l'espace enfant, ainsi que les actions principales attendues par le cahier des charges.

Ces maquettes ont servi de base de réflexion pour vérifier que les fonctionnalités importantes étaient bien représentées : connexion, inscription, dashboard parent, gestion des tâches, validation ou refus d'une preuve, gestion de la famille, invitations, création d'un compte enfant depuis l'espace parent et vue enfant avec preuve de fin de tâche.

Les maquettes basse fidélité ont ensuite été reprises et améliorées sous forme de maquettes haute fidélité, avec une identité visuelle plus précise, des couleurs, une typographie et une organisation d'écran plus proche de l'application finale.

[Maquette en annexe]

3.5.2 Maquettes haute fidélité et identité visuelle

Les maquettes haute fidélité ont été réalisées à partir des maquettes basse fidélité afin de représenter plus précisément l'apparence prévue de l'application Kiddo. Elles permettent de visualiser les écrans principaux, la navigation, les formulaires, les listes de tâches, les actions disponibles et la séparation entre les rôles parent et enfant.

L'objectif visuel de Kiddo est de proposer une interface simple, douce et rassurante, adaptée à une application familiale liée au suivi des devoirs et des tâches des enfants. Les couleurs choisies restent volontairement claires et agréables afin de ne pas donner une impression trop administrative. Les tons doux permettent de distinguer les zones importantes sans surcharger l'interface.

Le logo Kiddo utilise une approche ludique afin d'évoquer l'univers de l'enfance et de la famille, tout en restant lisible. Le choix typographique suit la même logique : la police Fredoka est utilisée pour les éléments plus identitaires, comme le logo ou certains titres et boutons, tandis que Nunito est utilisée pour les textes et les contenus d'interface. Ces deux polices ont été choisies car elles donnent un aspect accessible et léger, tout en conservant une bonne lisibilité pour les parents comme pour les enfants.

Les maquettes haute fidélité couvrent les principaux écrans : page d'accueil, connexion, inscription parent/enfant, dashboard parent, ajout et modification d'une

tâche, page famille côté parent, création d'un compte enfant, vue enfant, page famille côté enfant et ajout d'une tâche personnelle par l'enfant.

[Maquette en annexe]

3.5.3 Parcours utilisateur parent

Le parcours parent commence par la connexion à l'application. Une fois connecté, le parent accède à un dashboard présentant les tâches de ses enfants. Les tâches sont organisées selon leur état, par exemple les tâches en cours et les tâches à valider. Le parent peut aussi trier les tâches par date d'échéance ou par ordre alphabétique.

Depuis ce dashboard, le parent peut ajouter une nouvelle tâche pour un enfant. Le formulaire de création permet de définir le titre, la description, la date d'échéance et l'enfant concerné. Il permet aussi de définir les droits des autres parents sur cette tâche : lecture seule, modification ou aucun accès. Cette partie répond au besoin du cahier des charges concernant les familles dans lesquelles plusieurs parents peuvent avoir des droits différents sur une même tâche.

Lorsqu'un enfant marque une tâche comme terminée et fournit une preuve, le parent peut consulter cette preuve puis valider ou refuser la tâche. Cette validation est une action importante, car elle influence l'état de la tâche et doit respecter les règles de cohérence métier prévues dans le projet.

Kiddo

Dashboard Famille Déconnexion

Toutes les tâches

Ajouter une tâche

Trier par : Date d'échéance
Date d'échéance
Ordre alphabétique (titre)

En cours - 2

Exposé sciences
Lucas Echéance : 20 mai partagé

Ranger sa chambre
Emma Echéance : 22 mai privé

A valider - 2

Devoir de maths - ch. 7
Lucas Echéance : 12 mai Voir preuve ✓ ✕

Français - conjugaison des verbes
Emma Echéance : 14 mai Voir preuve ✓ ✕

2026 Kiddo - ETML TPI

The image shows two side-by-side screenshots of the Kiddo application interface. Both screenshots have a top navigation bar with 'Kiddo', 'Dashboard', 'Famille', and 'Déconnexion' links.

The left screenshot is titled 'Nouvelle tâche' (New task) and features a 'Créer la tâche' button. It contains the following fields:

- Titre de la tâche ***: A text input field with 'Devoir histoire' entered.
- Date d'échéance**: A date picker showing '20/05/2026'.
- Description**: A text area with 'Lire le chapitre 2 et faire les exercices 1 à 3'.
- Enfant concerné ***: A dropdown menu with 'Emma' selected and 'Lucas' as an option.
- Gestion des droits par co-parent**: A section with a dropdown for 'Jean Dupont' (Co-parent) and a sub-dropdown for 'Lecture seule' (Read only), with 'Modification' and 'Aucun accès' (No access) as other options.

The right screenshot is titled 'Modification d'une tâche' (Modify a task) and features an 'Enregistrer' (Save) button. It contains the same fields as the left screenshot, but with the 'Enregistrer' button instead of 'Créer la tâche'.

3.5.4 Gestion de la famille et des comptes enfants

La page famille côté parent permet de consulter les membres du groupe familial, séparés entre parents et enfants. Elle permet aussi de gérer les invitations applicatives et de créer directement un compte enfant.

Suite à un retour du chef de projet, la création d'un compte enfant est prévue directement depuis l'espace parent. Ce choix évite qu'un enfant crée un compte isolé sans rattachement à une famille. Lorsqu'un parent crée un compte enfant, celui-ci est automatiquement ajouté à la famille du parent.

Les invitations restent utilisées pour ajouter un autre compte existant au groupe familial, notamment un autre parent. L'invitation est visible dans l'espace Kiddo de l'utilisateur invité (dans l'onglet famille) et peut être acceptée ou refusée depuis l'application, sans utilisation d'e-mail.

The image shows a screenshot of the Kiddo application interface, specifically the 'Famille Jeanne Dupont' page. The top navigation bar includes 'Kiddo', 'Dashboard', 'Famille', and 'Déconnexion'.

The page title is 'Famille Jeanne Dupont' with '4 membres' (4 members) below it.

The page is divided into two main sections:

- Parents**: A list of family members. It shows 'Jeanne Dupont' with the role 'vous' (you) and 'Jean Dupont' with the role 'Retirer' (Remove).
- Enfants**: A list of family members. It shows 'Lucas' and 'Emma', both with the role 'Retirer' (Remove).

On the right side of the page, there is a section titled 'Invitations en attente' (Pending invitations). It shows an invitation from 'Tante' (Aunt) with the role 'Parent' and the text 'L'invité verra la notification dans son espace Kiddo.' (The invitee will see the notification in their Kiddo space). There is an 'Annuler' (Cancel) button next to it.

Below the invitations, there is a section titled 'Inviter un nouveau membre' (Invite a new member). It includes a text input field for 'Nom d'utilisateur Kiddo' (Kiddo username) with the example 'ex. john.doe' and an 'Envoyer l'invitation' (Send invitation) button.

At the bottom of the page, there is a link 'Créer un compte enfant' (Create a child account) with a plus icon. Below it, a message states: 'Un compte enfant créé par vous est automatiquement ajouté à votre famille.' (A child account created by you is automatically added to your family).

The footer of the page reads '2026 Kiddo - ETML TPI'.

Kiddo

Dashboard

Famille

Déconnexion

Créer un compte enfant - Famille Jeanne Dupont

[Retour](#)

Kiddo

Suivi des devoirs en famille

Compte Enfant

Nom d'utilisateur de l'enfant

john.doe

Nom de l'enfant

John Doe

Mot de passe

Confirmer mot de passe

Créer le compte enfant

2025 Kiddo - ETML TPI

3.5.5 Parcours utilisateur enfant

Le parcours enfant est volontairement plus simple que le parcours parent. Après connexion, l'enfant accède à la page "Mes tâches", où il voit uniquement les tâches qui lui sont attribuées. Pour chaque tâche, l'interface indique la date d'échéance et le statut.

L'enfant peut consulter le détail d'une tâche, ajouter une preuve sous forme de texte ou de fichier (ou photo directement), puis marquer la tâche comme terminée. La tâche passe alors en attente de validation, ce qui indique qu'un parent doit encore accepter ou refuser la preuve fournie.

L'enfant peut également créer une tâche personnelle. Dans ce cas, il peut définir la visibilité de cette tâche en choisissant les parents avec lesquels elle est partagée. Cette séparation permet de respecter la règle selon laquelle l'enfant ne peut modifier ou supprimer que les tâches qu'il a lui-même créées.

Kiddo

Mes tâchesFamilleDéconnexion

Mes tâchesAjouter une tâche personnelle

Tâche	Echeance	Statut
Devoir de maths - ch. 7	lundi 12 mai	à valider
Lecture chapitre 3	mardi 13 mai	à faire

Description de la tâche :

Faire exercices 1 à 5 du chapitre 7

Charger un fichier

Prendre une photo



J'ai fait quoi ? (preuve texte)

J'ai fait ceci et cela...

Marquer comme tâche terminée

En attente de validation

Devoir de maths - ch. 7

Terminé ✓
Preuve envoyée ✓

2026 Kiddo - ETML TPI

Kiddo

Mes tâches

Famille

Déconnexion

Famille

Invitations reçues - 1

Tante souhaite vous ajouter au groupe Famille Martin.
Acceptez-vous ?

Accepter

Refuser

Famille actuelle

Famille Dupont

Parents :

- Jeanne Dupont
- Jean Dupont

Enfants :

- Moi
- Emma

2026 Kiddo - ETML TPI

Kiddo

Dashboard

Famille

Déconnexion

Nouvelle tâche

Créer la tâche

Titre de la tâche *

Faire le lit

Date d'échéance

jj / mm / aaaa

Description

Ne pas oublier de faire le lit le matin.

Visibilité

☒ Jeanne Dupont

☒ Jean Dupont

2026 Kiddo - ETML TPI

3.5.6 Architecture générale

L'application Kiddo est conçue comme une application web séparée en trois parties principales : une interface frontend, une API backend et une base de données SQL.

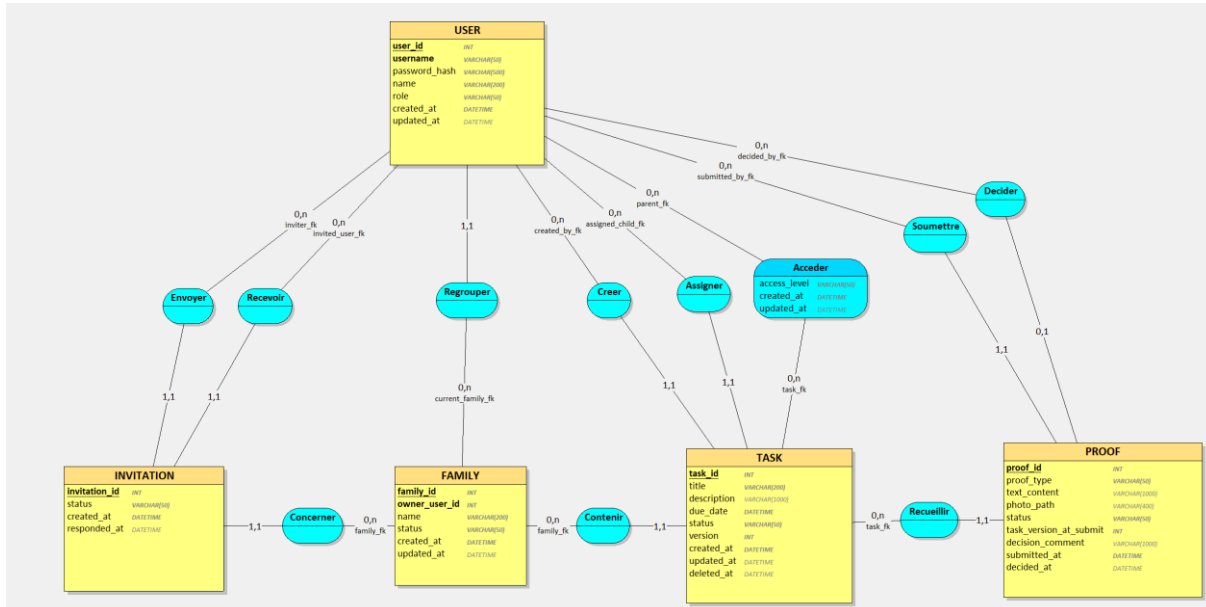
Le frontend est prévu avec Vue.js et Vite. Il sert à présenter les vues parent et enfant, les formulaires, les listes de tâches, les preuves et les invitations. Il communique avec le backend au moyen de requêtes HTTP.

Le backend est prévu avec AdonisJS. Il contient les règles métier principales de l'application : authentification, gestion des rôles, validation des entrées, contrôle des droits d'accès, gestion des tâches, preuves, invitations et règles de cohérence. Les contrôles sensibles sont réalisés côté backend, car l'interface frontend peut être contournée par des requêtes directes à l'API.

La base de données est prévue avec MySQL. Elle permet de stocker les utilisateurs, les familles, le rattachement courant des utilisateurs à une famille, les tâches, les preuves, les invitations et les droits d'accès aux tâches. Le choix d'une base relationnelle est adapté au projet, car les données sont fortement liées entre elles : un parent appartient à une famille, un enfant reçoit des tâches, une tâche peut avoir plusieurs preuves, et les droits d'accès doivent être vérifiables de manière fiable.

Cette architecture permet de séparer les responsabilités : le frontend gère l'affichage et l'expérience utilisateur, le backend applique les règles métier et la sécurité, et la base de données garantit la persistance et l'intégrité des données.

3.5.7 Choix de conception du modèle de données



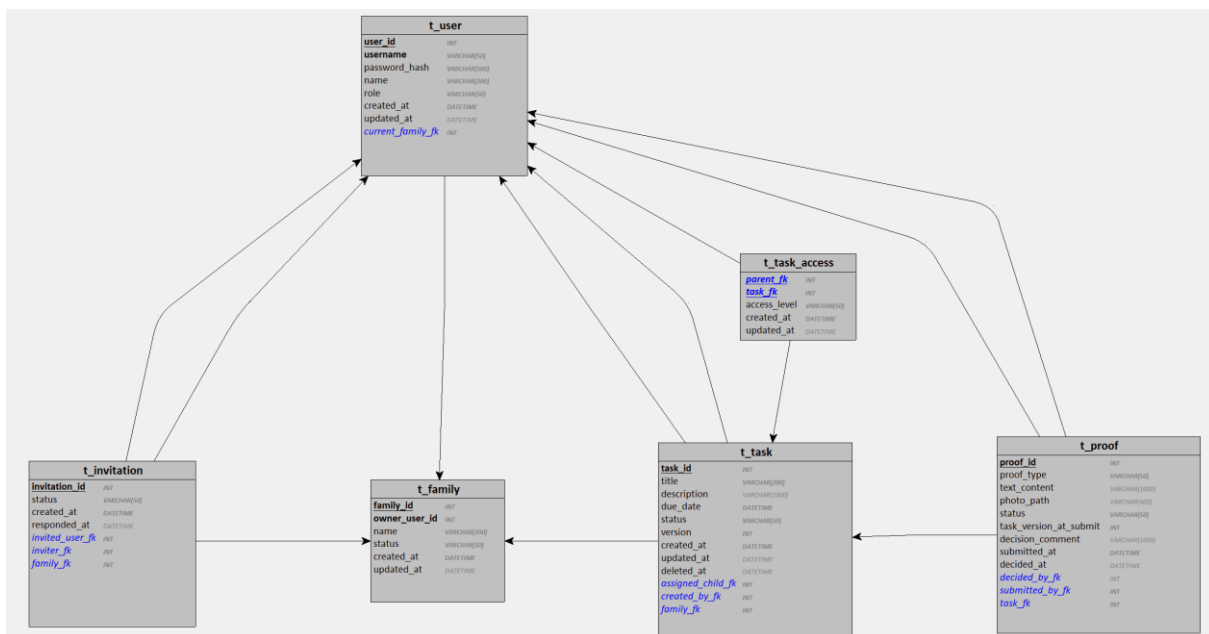
Le modèle de données a d'abord été réalisé avec Looping afin de représenter les entités principales et les relations métier du projet. Il a servi de base de conception pour identifier les utilisateurs, les familles, les rôles parent/enfant, les tâches, les preuves, les invitations et les droits d'accès aux tâches.

Le modèle présenté dans le diagramme ne doit cependant pas être considéré comme la version SQL définitive. Il présente donc une vue de conception, mais la

structure réellement appliquée et testée est celle du script SQL final. Lors de la préparation du script `setup_database.sql`, certaines limites sont apparues dans la traduction directe du modèle généré, notamment autour des relations entre les utilisateurs, les familles, les invitations et les droits d'accès. Le modèle a donc été repris manuellement afin d'obtenir un schéma plus cohérent avec l'implémentation réelle.

Le fichier `setup_database.sql` constitue la version de référence pour la base de données finale. Il reprend les tables prévues par le modèle, mais ajoute les éléments nécessaires à l'intégrité des données : clés étrangères, contraintes de cohérence, statuts contrôlés, champ de version pour les conflits métier et index utiles pour les recherches fréquentes.

Ce choix permet de conserver une base de conception compréhensible tout en disposant d'un schéma SQL réellement exploitable, contrôlé et défendable pour l'application.



3.5.8 Modèle relationnel retenu

Le choix retenu est un modèle relationnel, car les données du projet sont structurées et fortement liées entre elles. Une famille regroupe plusieurs utilisateurs, une tâche appartient à une famille, une tâche est assignée à un enfant, une preuve est liée à une tâche et les droits d'accès permettent de définir quels parents peuvent consulter ou modifier une tâche.

Ce type de structure est adapté à une base de données SQL, car il permet de représenter les relations à l'aide de clés primaires, de clés étrangères et de contraintes d'intégrité.

Afin de limiter la complexité du projet et d'éviter les incohérences liées à plusieurs appartenances familiales simultanées, le modèle retenu impose qu'un utilisateur soit rattaché à une seule famille courante à la fois. Cette relation est représentée par le champ `current_family_fk` dans la table `t_user`.

Lors de la création d'un compte, une famille personnelle est créée automatiquement pour l'utilisateur. Cette famille est identifiée par le champ `owner_user_id` dans la table `t_family`.

Comme `t_user` possède aussi une référence vers `t_family` avec `current_family_fk`, il existe une dépendance circulaire entre les deux tables. Pour résoudre ce problème proprement, le script crée d'abord `t_family` avec `owner_user_id` nullable, puis crée `t_user`, et ajoute ensuite la clé étrangère `owner_user_id` avec `ALTER TABLE`. Ce fonctionnement permet de conserver l'intégrité référentielle sans bloquer la création du schéma SQL.

```
53
54 CREATE TABLE t_family(
55     family_id INT AUTO_INCREMENT,
56     owner_user_id INT NULL,
57     name VARCHAR(200) NOT NULL,
58     status VARCHAR(50) NOT NULL,
59     created_at DATETIME NOT NULL,
60     updated_at DATETIME NULL,
61
62     PRIMARY KEY(family_id),
63
64     CONSTRAINT chk_family_status
65     CHECK (status IN ('active', 'archived'))
66 );
67
```

```
135
136 ALTER TABLE t_family
137     ADD CONSTRAINT uq_family_owner UNIQUE(owner_user_id),
138     ADD CONSTRAINT fk_family_owner
139     FOREIGN KEY(owner_user_id)
140     REFERENCES t_user(user_id);
141
```

Si un utilisateur accepte une invitation pour rejoindre une autre famille, le système vérifie d'abord que sa famille personnelle ne contient pas d'autres membres actifs. Si c'est le cas, l'utilisateur est déplacé vers la famille invitante et sa famille personnelle peut être archivée. S'il quitte ensuite cette famille, sa famille personnelle est réactivée ou recrée automatiquement. Ce choix permet de répondre au besoin d'invitation tout en conservant un modèle simple et maîtrisable.

Les tables prévues pour le modèle de données sont les suivantes :

t_user : représente un compte parent ou enfant.

t_family : représente une famille courante ou personnelle.

t_invitation : représente une invitation applicative envoyée par un parent.

t_task : représente un devoir ou une tâche à réaliser.

t_proof : représente une preuve texte ou photo envoyée par l'enfant.

t_task_access : représente les droits d'accès des parents sur une tâche.

La seule table associative du modèle est t_task_access. Elle est nécessaire car une tâche peut être accessible à plusieurs parents, et un parent peut avoir accès à plusieurs tâches. Cette table permet de définir un niveau d'accès précis : aucun accès, lecture seule ou modification.

En pratique, l'absence de droit ou un niveau none est traité comme une absence d'accès. Le niveau none permet cependant de représenter explicitement un refus d'accès si cette information doit être conservée.

3.5.9 Rôles, droits d'accès et cohérence métier

Deux rôles principaux sont prévus dans l'application : parent et child.

Un parent peut gérer les tâches liées à sa famille courante. Il peut créer une tâche pour un enfant, modifier ou supprimer une tâche selon les règles prévues, inviter un autre utilisateur dans la famille et valider ou refuser une preuve envoyée par un enfant.

Un enfant peut consulter uniquement les tâches qui lui sont attribuées. Il peut marquer une tâche comme terminée et fournir une preuve sous forme de texte ou de photo. Il peut également créer ses propres tâches, mais il ne peut modifier ou supprimer que les tâches qu'il a lui-même créées.

Les droits d'accès des parents sur les tâches sont gérés par la table t_task_access. Cette table contient le champ access_level, qui permet de distinguer trois niveaux d'accès :

none : le parent ne peut pas accéder à la tâche.

read : le parent peut consulter la tâche.

write : le parent peut modifier la tâche et, selon l'état de celle-ci, valider ou refuser une preuve.

Les contrôles de droits sont appliqués côté backend. L'interface frontend peut masquer certaines actions, mais elle ne doit pas être considérée comme une protection suffisante. Chaque action sensible est donc vérifiée par l'API avant d'être acceptée.

Pour garantir la cohérence des données, les tâches et les preuves utilisent des statuts contrôlés. Une tâche ne doit pas pouvoir se retrouver dans deux états incompatibles, par exemple validée et refusée en même temps. Les actions critiques comme une modification après validation, une suppression concurrente ou une validation basée sur une ancienne version de la tâche doivent être empêchées ou traitées de manière déterministe.

La table t_task contient un champ version, utilisé pour détecter les actions réalisées sur une ancienne version d'une tâche. La table t_proof contient le champ task_version_at_submit, qui permet de vérifier que la preuve envoyée correspond bien

à la version de la tâche au moment de la soumission. Si une action repose ensuite sur une version obsolète, le backend peut refuser l'action avec une erreur de conflit.

La validation ou le refus d'une preuve est stocké directement dans `t_proof` grâce aux champs `status`, `decided_by_fk`, `decision_comment` et `decided_at`. Une table séparée de validation n'a pas été retenue, car le cahier des charges ne demande pas d'historique détaillé des décisions. Ce choix simplifie le modèle tout en permettant de savoir si une preuve est en attente, validée ou refusée.

3.5.10 Contraintes SQL et valeurs contrôlées

Le script `setup_database.sql` utilise plusieurs contraintes SQL afin d'éviter les incohérences directement au niveau de la base de données.

Les contraintes de clé étrangère FOREIGN KEY garantissent qu'une donnée liée existe réellement dans la table de référence. Par exemple, une invitation ne peut pas être liée à un utilisateur invité inexistant, à un utilisateur expéditeur inexistant ou à une famille inexistante. Cela évite les données orphelines.

Les contraintes CHECK limitent certaines valeurs possibles. Par exemple, un utilisateur ne peut avoir que le rôle `parent` ou `child`, une tâche ne peut avoir que les statuts `todo`, `submitted`, `validated` ou `refused`, et une preuve ne peut être que `pending`, `validated` ou `refused`.

Les valeurs contrôlées sont les suivantes :

t_user.role : `parent`, `child`

t_family.status : `active`, `archived`

t_invitation.status : `pending`, `accepted`, `refused`, `cancelled`

t_task.status : `todo`, `submitted`, `validated`, `refused`

t_proof.status : `pending`, `validated`, `refused`

t_proof.proof_type : `text`, `photo`, `text_photo`

t_task.access.access_level : `none`, `read`, `write`

Ces contraintes ne remplacent pas la validation côté backend. Le backend valide les valeurs envoyées par l'utilisateur afin de retourner des erreurs compréhensibles à l'interface. Le script SQL ajoute une protection supplémentaire afin d'empêcher l'enregistrement de valeurs incohérentes directement dans la base de données.

Certaines règles métier plus précises restent contrôlées côté backend. Par exemple, une clé étrangère permet de vérifier qu'un utilisateur existe, mais c'est le backend qui vérifie si cet utilisateur a le bon rôle et le droit d'effectuer une action donnée.

Le script ajoute également des index sur les colonnes utilisées dans les recherches fréquentes, par exemple les tâches d'une famille par statut, les tâches triées par date d'échéance, les preuves liées à une tâche et les invitations reçues par un utilisateur. Ces index permettent d'améliorer les performances sur les requêtes principales de l'application.

3.6 Planification et suivi

3.7 Réalisation

La phase de réalisation a consisté à transformer les choix de conception en une première version fonctionnelle du backend. Le développement a d'abord été concentré sur la base de données, les migrations, l'authentification et les règles métier principales, afin de disposer d'une API stable avant de commencer l'intégration frontend.

L'objectif était de valider rapidement les parties les plus sensibles du projet : les comptes utilisateurs, les familles, les rôles parent/enfant, les tâches, les preuves, les invitations, les droits d'accès et la cohérence des données. Ces éléments sont centraux dans le cahier des charges et influencent directement le frontend et les tests.

3.7.1 Mise en place de l'environnement backend

Le backend a été initialisé avec AdonisJS en mode API, avec une configuration MySQL et une authentification basée sur des access tokens. L'environnement local utilise Docker pour exécuter MySQL et phpMyAdmin. Cette configuration permet de travailler localement avec une base de données proche de celle prévue pour la suite du projet.

Le fichier .env du backend contient les variables de connexion locales à la base de données. Ce fichier n'est pas versionné afin d'éviter l'exposition d'informations sensibles. La connexion à MySQL est configurée dans config/database.ts, et le backend communique avec la base via Lucid ORM et des requêtes contrôlées.

3.7.2 Implémentation de la base de données et des migrations

Le script setup_database.sql a été préparé à partir du modèle de données retenu. Il crée les tables principales, les clés primaires, les clés étrangères, les contraintes de cohérence et les index utiles. Ce script sert de référence pour la structure SQL finale.

En parallèle, les migrations AdonisJS ont été adaptées afin que l'environnement backend puisse reconstruire la base de données pendant le développement. Les migrations générées par défaut ont été modifiées, car AdonisJS crée initialement une table users, alors que le modèle Kiddo utilise une table t_user.

3.7.3 Authentification et sécurité backend

L'authentification a été implémentée avec des access tokens. Les routes principales sont les suivantes : inscription, connexion, récupération de l'utilisateur connecté et déconnexion. Les routes sensibles sont protégées par le middleware d'authentification.

Le hachage des mots de passe a été configuré avec Argon2id. Ce choix permet de ne jamais stocker les mots de passe en clair et répond à l'exigence de sécurité liée à la protection des comptes. Le modèle utilisateur AdonisJS a été adapté au schéma Kiddo afin d'utiliser la table t_user, le champ username comme identifiant et le champ password_hash pour le mot de passe haché.

Les entrées envoyées à l'API sont validées côté backend avec des validateurs. Les contrôles backend restent indispensables, car le frontend peut être contourné par des requêtes directes. Cette logique rejoint les exigences non fonctionnelles liées à la

validation des données, aux rôles et aux autorisations. Ton rapport mentionne déjà que les rôles et droits doivent être vérifiés côté backend dans les exigences non fonctionnelles.

3.7.4 Implémentation des fonctionnalités métier

Les premières fonctionnalités métier du backend ont été implémentées autour des familles, des tâches, des preuves et des invitations.

Pour la gestion familiale, un parent peut consulter les membres de sa famille, créer directement un compte enfant et inviter un autre utilisateur existant. Lorsqu'un compte enfant est créé depuis l'espace parent, il est automatiquement rattaché à la famille du parent. Une famille personnelle est également prévue afin que l'utilisateur ne se retrouve jamais sans famille courante. Cette logique correspond à la conception déjà décrite dans la partie famille du rapport, qui précise que la création d'un compte enfant depuis l'espace parent évite un compte isolé sans rattachement familial.

La gestion des tâches permet à un parent de créer, consulter, modifier et supprimer logiquement une tâche. Une tâche est liée à une famille, à un enfant assigné et à un parent créateur. Le parent créateur obtient automatiquement un accès en modification. Les autres parents présents dans la famille reçoivent le niveau d'accès défini lors de la création de la tâche : aucun accès, lecture seule ou modification.

La gestion des preuves permet à un enfant de soumettre une preuve sous forme de texte, de photo ou des deux. Lorsqu'une preuve est envoyée, la tâche passe en statut submitted. Un parent disposant du droit nécessaire peut ensuite valider ou refuser la preuve. La décision met à jour à la fois la preuve et la tâche.

La gestion des invitations permet à un parent d'inviter un autre utilisateur dans sa famille. L'utilisateur invité peut accepter ou refuser l'invitation depuis l'application. Lorsqu'un parent quitte une famille rejointe, il est automatiquement rattaché à sa famille personnelle. Un parent ne peut pas quitter sa propre famille personnelle.

3.7.5 Corrections et ajustements après tests Bruno

Des tests manuels avec Bruno ont été réalisés au fur et à mesure de l'implémentation. Ces tests ont permis de vérifier les routes principales, mais aussi d'identifier certaines règles métier à ajuster.

Un premier ajustement a concerné le retrait d'un enfant d'une famille. Au départ, le retrait était bloqué dès que l'enfant possédait une tâche dans la famille. Après test, cette règle s'est révélée trop stricte, car une tâche déjà validée est considérée comme terminée. La condition a donc été corrigée afin de bloquer uniquement les tâches non terminées, c'est-à-dire les tâches en statut todo, submitted ou refused.

▼ Kiddo
POST Login (parent)
POST Login (child)
POST Validate Proof
POST Create Tasks
POST Create New User (parent) Account
GET All User's Tasks
POST Leave User's Own Family (not possible)
POST Other Member of Family (parent) Leaves
POST Invite User to Family
POST Post Proof
POST Create Child Account in Parent Family
GET All User's Family Members
POST User Accepts Invitation
GET See Proof
GET Get invitations of User
PUT Update Tasks
DEL Delete Tasks
DEL Remove Family Member

Un autre point observé concerne les droits d'accès des parents qui rejoignent une famille après la création d'une tâche. Dans l'implémentation actuelle, les droits sont attribués au moment de la création de la tâche. Un parent qui rejoint la famille après coup ne reçoit donc pas automatiquement l'accès aux anciennes tâches. Ce choix est conservé afin d'éviter de modifier rétroactivement les droits d'accès et de préserver la confidentialité des tâches créées avant son arrivée.

Les tests Bruno ont aussi permis de valider les flux principaux : inscription, connexion, création d'un compte enfant, création de tâche, soumission d'une preuve, validation ou refus d'une preuve, invitation familiale, retrait d'un membre et départ d'un parent d'une famille rejointe.

3.7.6 Données initiales avec init_database.sql

Le fichier init_database.sql a été préparé afin de disposer d'un jeu de données de test cohérent après la création du schéma SQL. Contrairement à setup_database.sql, qui définit la structure de la base, init_database.sql insère des utilisateurs, familles, tâches, preuves, invitations et droits d'accès.

Ces données permettent de tester rapidement les parcours principaux sans devoir recréer manuellement les mêmes comptes à chaque réinitialisation de la base. Elles couvrent notamment un parent principal, un co-parent, des enfants, des tâches en différents statuts et des droits d'accès distincts. Ce choix facilite les tests manuels avec Bruno, les vérifications frontend et la future automatiser des tests.

user_id	username	password_hash	name	role	created_at	updated_at	current_family_fk
1	nelson	\$argon2id\$v=19\$m=65536,t=3,p=4\$dUFZaayHQxvXLfsHRA1...	Nelson Almeida	parent	2026-05-20 08:30:00	NULL	1
2	jeanne	\$argon2id\$v=19\$m=65536,t=3,p=4\$1NQwUtETuOBuovklm...	Jeanne Dupont	parent	2026-05-20 08:35:00	2026-05-20 09:00:00	1
3	lucas	\$argon2id\$v=19\$m=65536,t=3,p=4\$JShdGt1XbcKATcjQYCF...	Lucas Almeida	child	2026-05-20 08:40:00	NULL	1
4	emma	\$argon2id\$v=19\$m=65536,t=3,p=4\$gbPNyK7IPXMGpcF/Tkh...	Emma Almeida	child	2026-05-20 08:45:00	NULL	1
5	marie	\$argon2id\$v=19\$m=65536,t=3,p=4\$alT0j/H4Pk4UkQFxcrl...	Marie Martin	parent	2026-05-20 08:50:00	NULL	5

3.7.7 Implémentation du frontend

Le frontend a été initialisé avec Vue.js et Vite. La structure a été organisée autour de vues, composants, services API et store d'authentification. Axios est utilisé pour communiquer avec le backend, et le token d'authentification est ajouté automatiquement aux requêtes lorsque l'utilisateur est connecté.

Les premières vues mises en place concernent l'authentification, la navigation, le dashboard parent, le détail d'une tâche, la création et la modification des tâches, ainsi que l'écran "Mes tâches" côté enfant. Le frontend applique une séparation

visible entre les rôles parent et enfant : après connexion, un parent est redirigé vers le dashboard parent, tandis qu'un enfant arrive sur sa liste de tâches.

Les contrôles sensibles restent volontairement côté backend. Le frontend masque certaines actions selon le contexte, par exemple la modification d'une tâche validée, mais l'API conserve la responsabilité finale de refuser les actions non autorisées.

[à mettre capture dashboard parent]

[capture consultation d'une preuve et décision parent]

[capture vue enfant]

(...)

3.7.8 Déploiement Azure

Un déploiement Azure intermédiaire a été réalisé avant la fin complète du développement **afin de valider l'infrastructure technique**. L'objectif n'était pas de figer la version finale, mais de vérifier que les différents services pouvaient fonctionner ensemble : **base de données Azure MySQL, backend Azure App Service et frontend Azure Static Web Apps**.

[captures]

La base de données a été déployée sur Azure Database for MySQL Flexible Server. Les scripts `setup_database.sql` et `init_database.sql` ont été exécutés afin de créer le schéma et d'insérer les données de test.

[captures]

Le backend a ensuite été déployé sur Azure App Service avec des variables d'environnement dédiées à la production. Le frontend a été déployé sur Azure Static Web Apps et configuré pour appeler l'URL du backend Azure.

[captures]

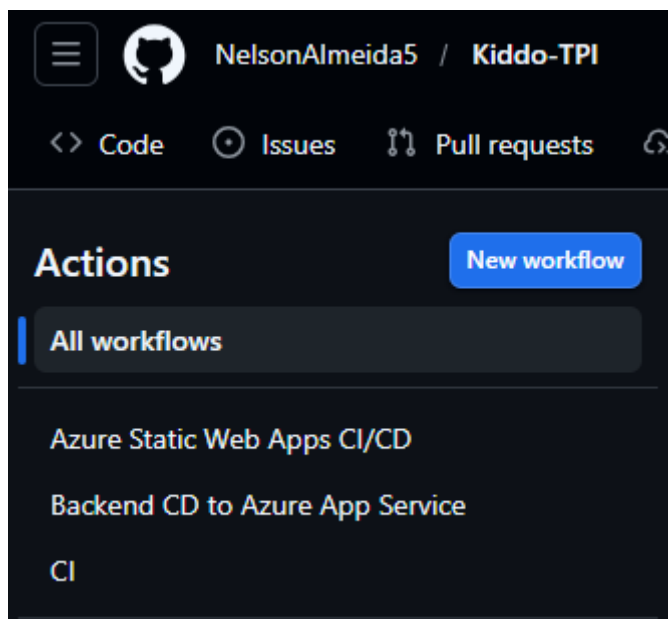
Cette étape a permis de vérifier tôt les problèmes potentiels de configuration, notamment la connexion sécurisée à MySQL, les variables d'environnement, les URLs entre frontend et backend et le comportement de l'application hors environnement local.

...[à développer par rapport aux démarches]

3.7.9 Mise en place du CI/CD

Deux workflows de déploiement continu ont été mis en place avec GitHub Actions : un pour le backend et un pour le frontend. Le workflow backend construit l'application AdonisJS, prépare le dossier de build avec les dépendances de production et déploie l'archive sur Azure App Service. Le workflow frontend, généré via Azure Static Web Apps, construit l'application Vue/Vite et la déploie automatiquement sur Azure après un push sur la branche main.

Un premier workflow CI séparé a également été ajouté afin de contrôler automatiquement la qualité technique du projet. Il exécute les vérifications principales du backend et du frontend, notamment le lint, le typecheck backend et les builds. **Les tests automatisés seront ensuite intégrés dans ce pipeline.**



3.8 Tests et validation

Les tests ont pour objectif de vérifier que l'application Kiddo respecte les exigences du cahier des charges et que les règles métier principales sont correctement appliquées. Une attention particulière est portée aux rôles parent/enfant, aux droits d'accès, aux tâches, aux preuves, aux validations et à la cohérence des données.

La stratégie de test privilégie les tests automatisés lorsque cela est réaliste dans le temps disponible du TPI, limité à 90 heures. Les tests manuels restent utilisés pour vérifier les aspects visuels, la navigation, le responsive et certains parcours utilisateur dans l'interface.

3.8.1 Stratégie et environnement de test

Système testé	Application web Kiddo
Frontend	Vue.js / Vite, interface utilisée par les parents et les enfants
Backend	AdonisJS / Node.js, API responsable des règles métier, validations et autorisations

Base de données	Base SQL MySQL contenant les comptes, familles, tâches, preuves et droits
Administration DB	phpMyAdmin utilisé en local pour vérifier les tables et les données de test
Données de test	Comptes fictifs parent/enfant, famille de test, tâches et preuves
Utilisateurs de test	Parent A, Parent B, Enfant A
Tests automatisés	Tests backend exécutés localement puis dans GitHub Actions
CI/CD	GitHub Actions pour lancer les tests avant la mise en production
Production	Application déployée sur Azure après validation

3.8.2 Tests prévus

ID	Type	Objectif	Exigence liée	Résultat attendu
TU-01	Unitaire	Vérifier qu'un utilisateur avec le rôle parent peut effectuer une action parent	ENF-03	L'action parent est autorisée
TU-02	Unitaire	Vérifier qu'un utilisateur avec le rôle enfant ne peut pas effectuer une action parent	ENF-03	L'action parent est refusée
TU-03	Unitaire	Vérifier qu'un champ obligatoire vide est refusé	ENF-04	La donnée invalide est refusée
TU-04	Unitaire	Vérifier qu'une valeur de statut valide est acceptée	ENF-07	Le statut valide est accepté
TU-05	Unitaire	Vérifier qu'une transition de statut interdite est refusée	ENF-07	L'état incohérent est empêché
TU-06	Unitaire	Vérifier qu'un parent sans droit de modification est refusé	EF-05 / ENF-03	L'action est refusée
TU-07	Unitaire	Vérifier qu'un enfant ne peut modifier que ses propres tâches	EF-09 / ENF-03	L'action non autorisée est refusée
TU-08	Unitaire	Vérifier qu'une tâche validée ne peut plus être modifiée directement	EF-11 / ENF-07	La modification directe est refusée
TU-09	Unitaire	Vérifier qu'un message d'erreur ne révèle pas d'information sensible	ENF-06	Le message reste contrôlé
TU-10	Unitaire	Vérifier qu'une donnée manipulée est refusée	ENF-04 / ENF-05	La donnée est refusée
TI-01	Intégration	Tester une séquence complète : tâche terminée avec preuve, validation parent, puis tentative de modification après validation	EF-06 / EF-08 / EF-11 / ENF-07	Le système empêche l'état incohérent et respecte les règles définies

3.8.3 Exécution automatique dans le CI/CD

Les tests automatisés seront exécutés dans le pipeline CI/CD avant la mise en production. Le pipeline devra installer les dépendances, lancer les tests disponibles et signaler une erreur si un test échoue.

Cette exécution automatique permet de vérifier que les règles principales restent valides avant le déploiement sur Azure.

3.8.4 Exécution et résultats des tests

(Cette section sera complétée pendant la phase d'exécution des tests. Elle contiendra les résultats obtenus, les éventuels échecs, les corrections appliquées et les preuves d'exécution, notamment dans le pipeline CI/CD.)

ID	Résultat attendu	Résultat obtenu	Statut	Correction si nécessaire

3.9 Livraison

3.10 Conclusion et bilan personnel

4 Références

Cahier des charges TPI : CDC_Kiddo_NelsonAlmeida.pdf, ETML Section informatique, 2026.

Critères d'évaluation TPI : Critères_Evaluation_NelsonAlmeida 1.pdf, procédure de qualification Informaticien CFC OrFo 2021.

OWASP Foundation. OWASP Top 10:2025. Consulté le 11 mai 2026 :

<https://owasp.org/Top10/2025/>

Documentation AdonisJS

<https://docs.adonisjs.com/>

5 Annexes

5.1 Journal de travail

5.2 Liste des livrables remis

Annexe 1 : Planification et Journal de travail

Annexe 2 : Tableau détaillé des risques et mesures de sécurité

Annexe 3 : Maquette basse fidélité

Annexe 4 : Maquette haute fidélité

5.3 Repos GitHub

<https://github.com/NelsonAlmeida5/Kiddo-TPI>

5.4 Table des illustrations